

State-Space Models

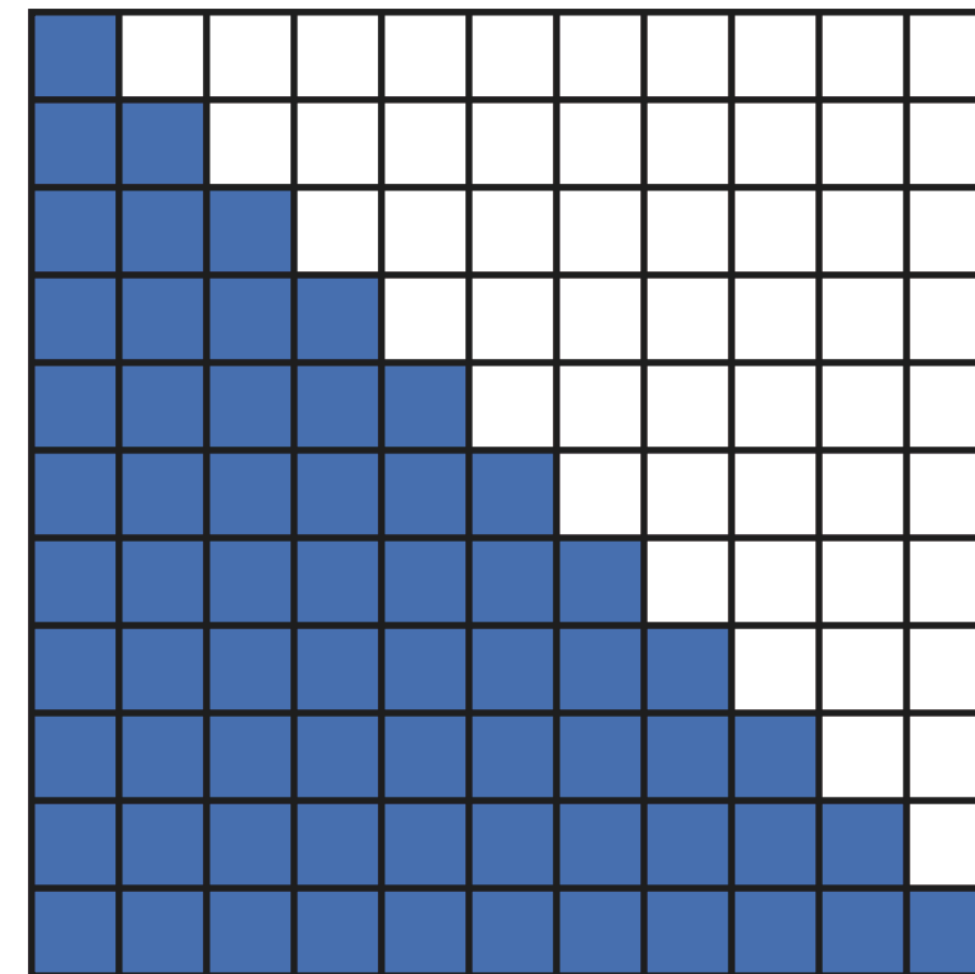
Agenda

- State Space Models
- RWKV

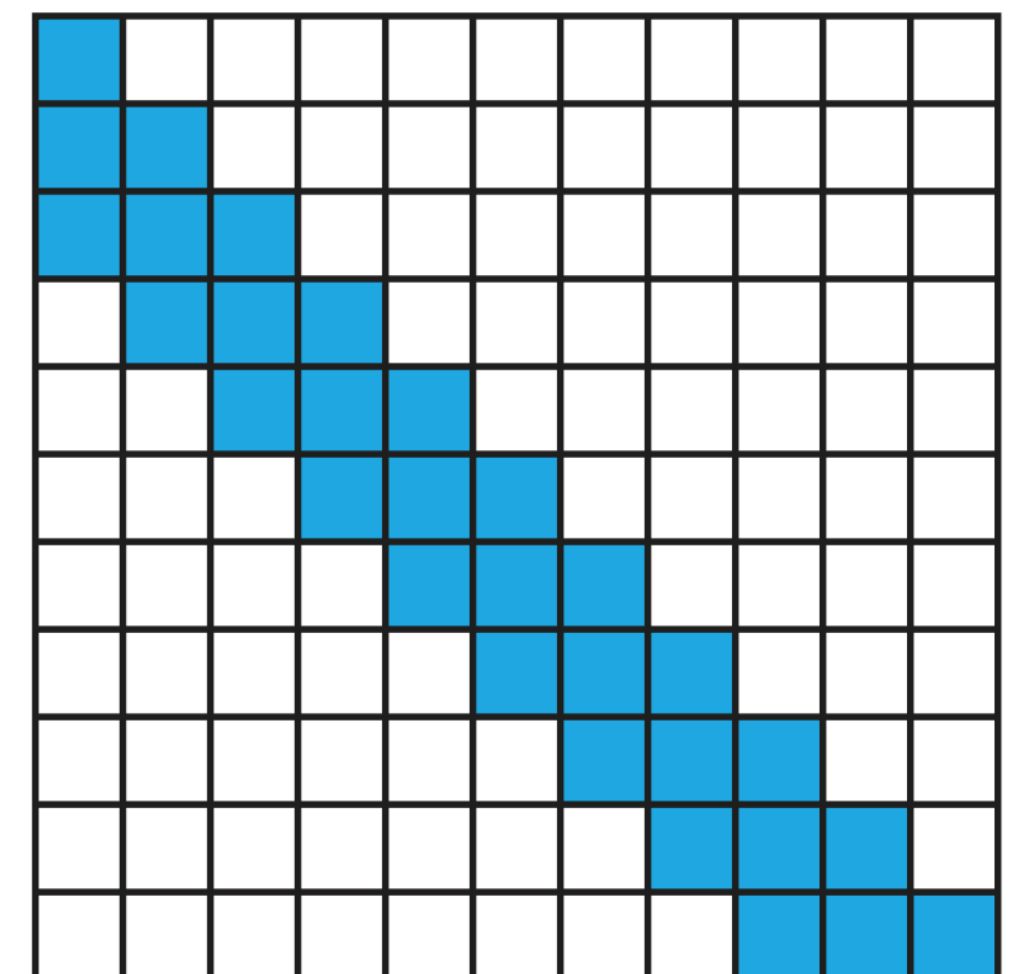
Quadratic Attention

- Vanila attention has quadratic complexity
- It makes Transformers hard to apply to long contexts
- Especially in the era on reasoning
- Sliding window attention (and others) is a solution, but we can do better

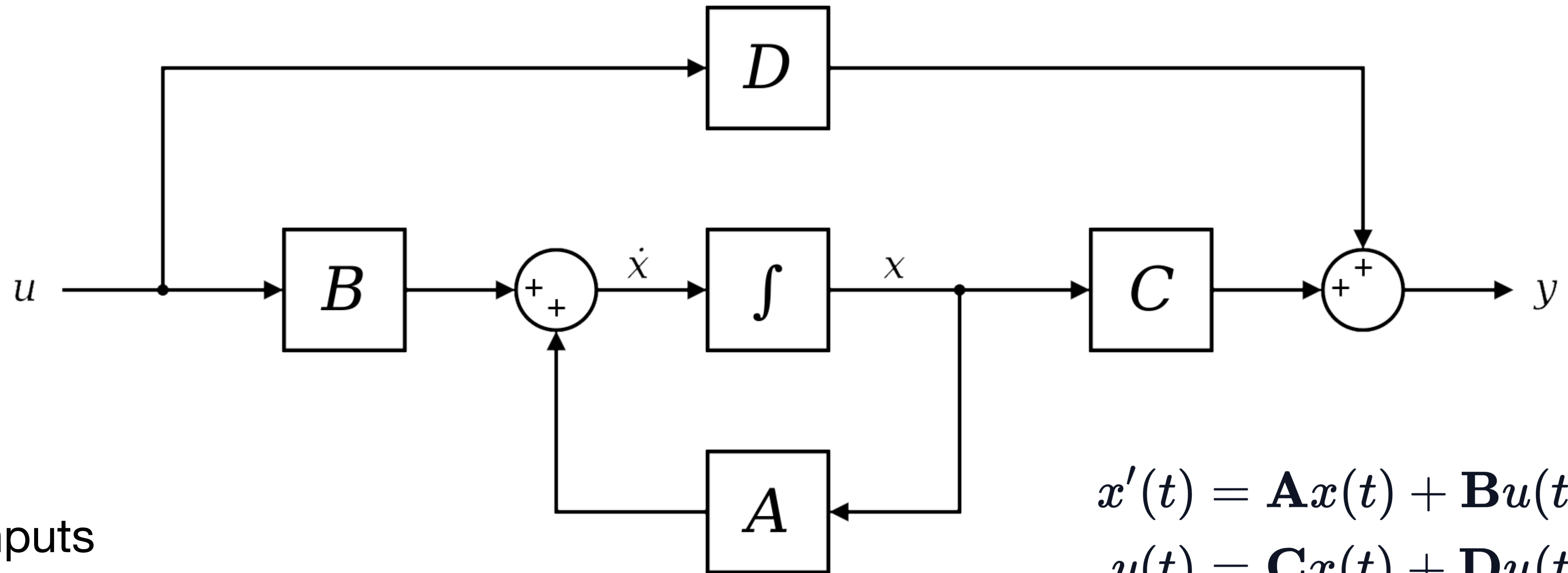
Linear Attention



Sliding Window



State Space Models



u is inputs

y is outputs

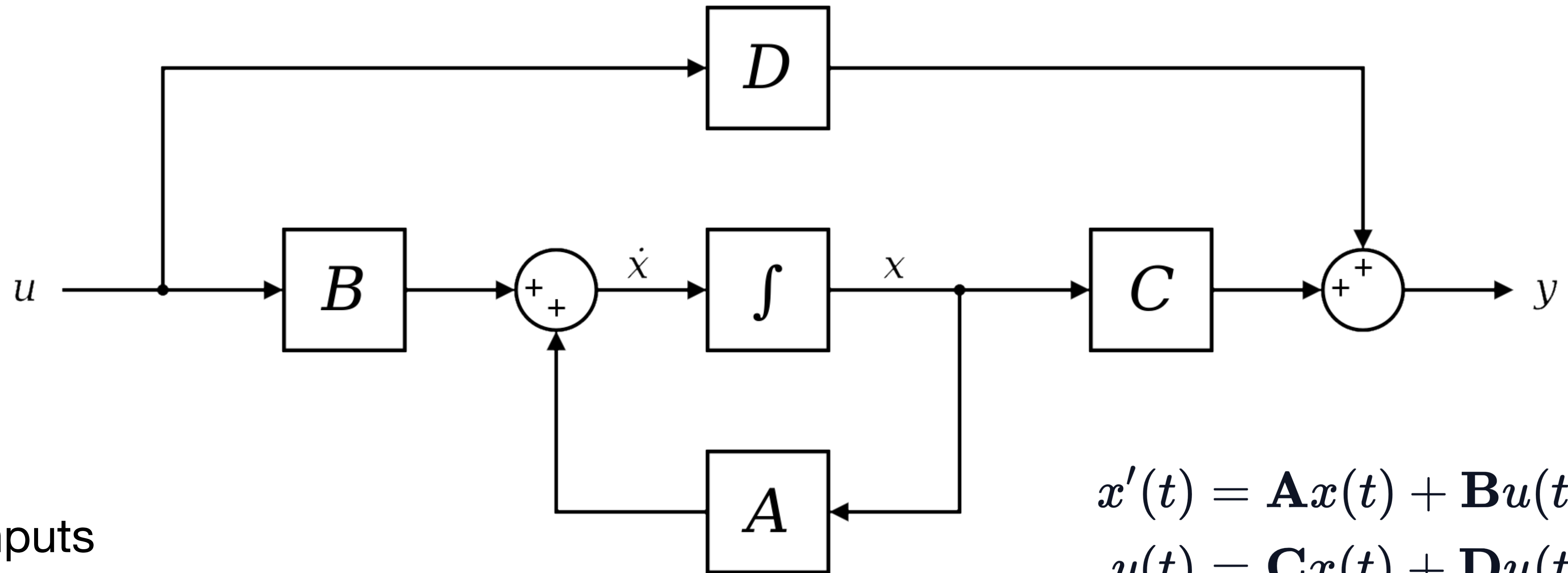
x is a hidden state (yes, like in RNNs)

$\dot{x}$ is the derivative

$$\dot{x}(t) = \mathbf{A}x(t) + \mathbf{B}u(t)$$

$$y(t) = \mathbf{C}x(t) + \mathbf{D}u(t)$$

State Space Models



u is inputs

y is outputs

x is a hidden state (yes, like in RNNs)

$\dot{x}$ is the derivative

$$\dot{x}(t) = \mathbf{A}x(t) + \mathbf{B}u(t)$$

$$y(t) = \mathbf{C}x(t) + \mathbf{D}u(t)$$

or

$$\dot{x} = \mathbf{A}x + \mathbf{B}u$$

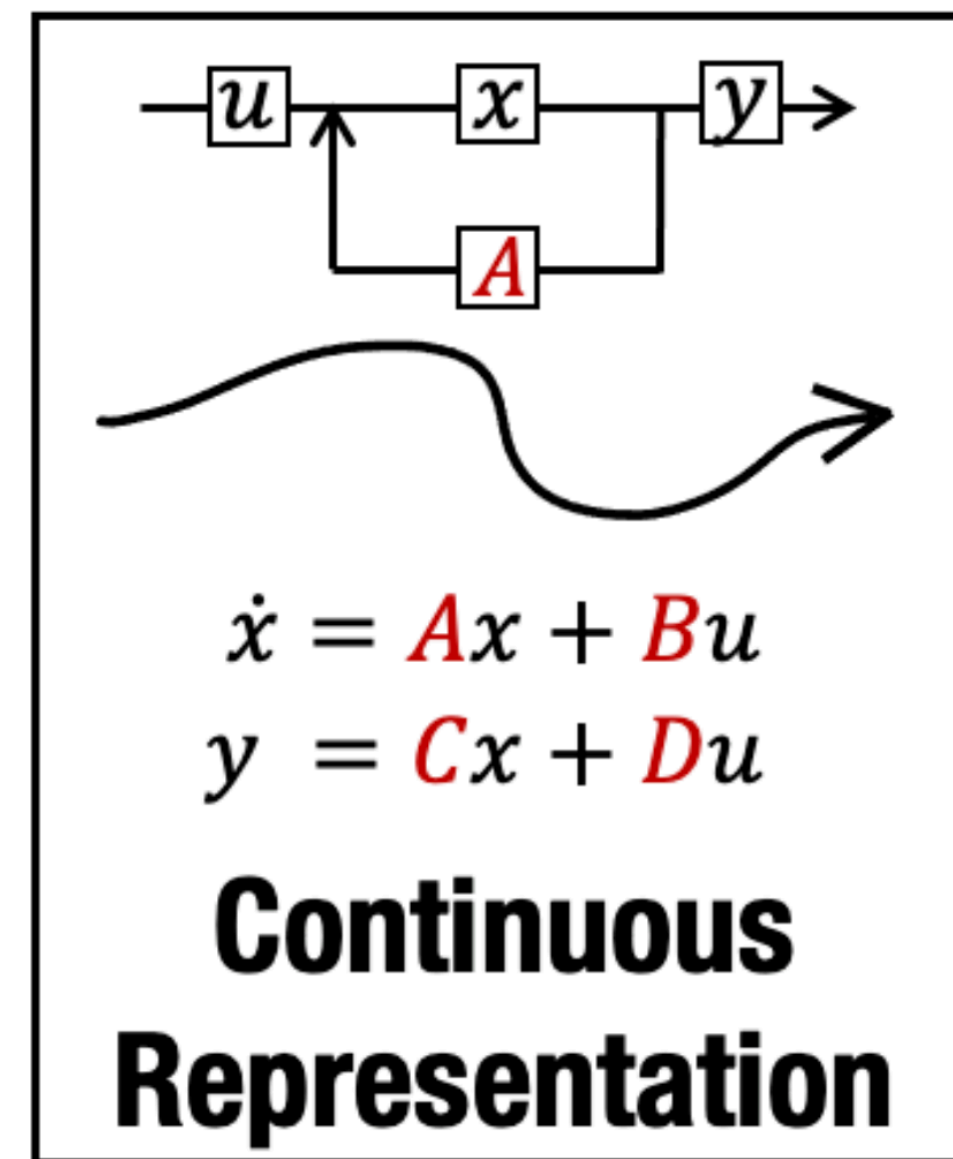
$$y = \mathbf{C}x$$

State Space Models

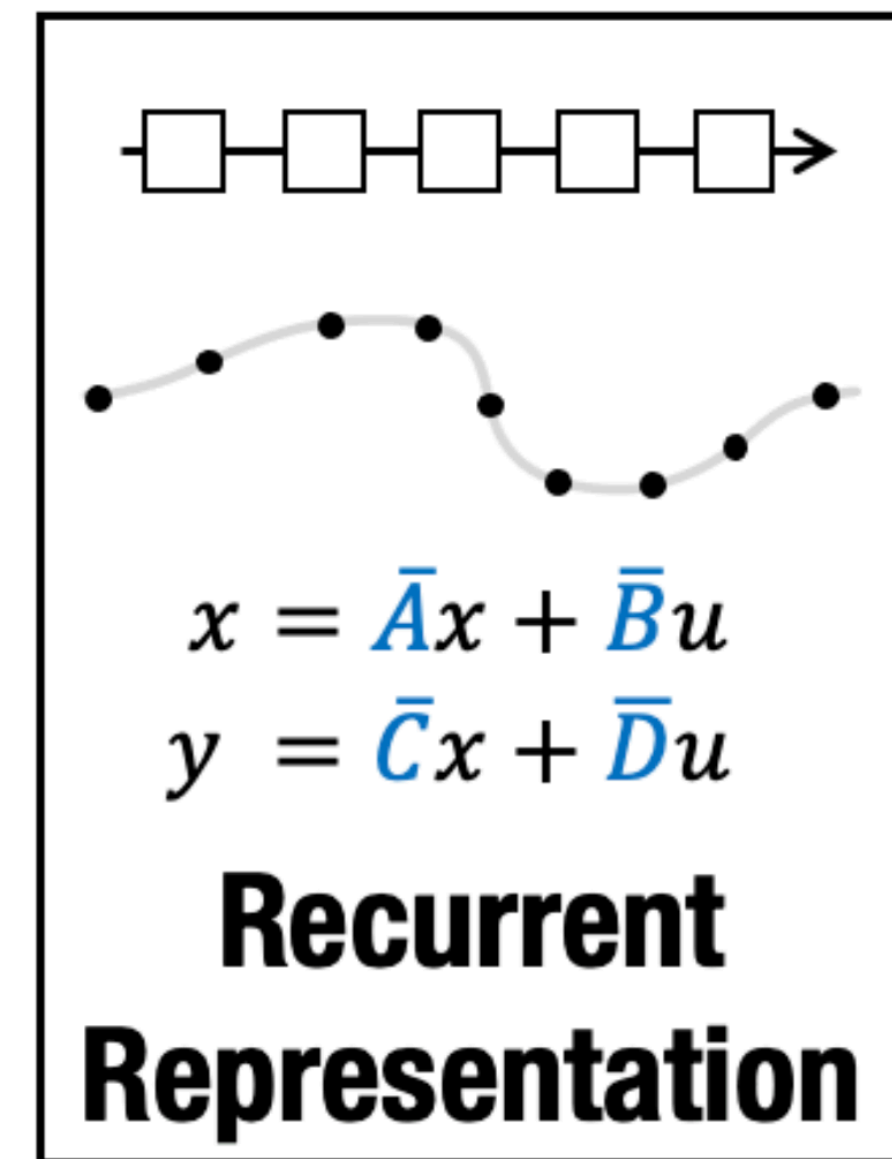
This system is continuous and should be discretized

$$\dot{x} = \mathbf{A}x + \mathbf{B}u$$

$$y = \mathbf{C}x$$



Discretize



Discretization

$$\dot{x} = \mathbf{A}x + \mathbf{B}u$$

$$y = \mathbf{C}x$$

Trapezoid method:
$$x_{n+1} - x_n = \frac{1}{2}\Delta(f(t_n) + f(t_{n+1})) \quad \Delta = t_{n+1} - t_n$$

$$x_{n+1} = x_n + \frac{\Delta}{2}(\mathbf{A}x_n + \mathbf{B}u_n + \mathbf{A}x_{n+1} + \mathbf{B}u_{n+1})$$

Discretization

$$\dot{x} = \mathbf{A}x + \mathbf{B}u$$

$$y = \mathbf{C}x$$

Trapezoid method:
$$x_{n+1} - x_n = \frac{1}{2}\Delta(f(t_n) + f(t_{n+1})) \quad \Delta = t_{n+1} - t_n$$

$$x_{n+1} = x_n + \frac{\Delta}{2}(\mathbf{A}x_n + \mathbf{B}u_n + \mathbf{A}x_{n+1} + \mathbf{B}u_{n+1})$$

$$\iff x_{n+1} - \frac{\Delta}{2}\mathbf{A}x_{n+1} = x_n + \frac{\Delta}{2}\mathbf{A}x_n + \frac{\Delta}{2}\mathbf{B}(u_{n+1} + u_n)$$

Discretization

$$x' = \mathbf{A}x + \mathbf{B}u$$

$$y = \mathbf{C}x$$

Trapezoid method:
$$x_{n+1} - x_n = \frac{1}{2}\Delta(f(t_n) + f(t_{n+1})) \quad \Delta = t_{n+1} - t_n$$

$$x_{n+1} = x_n + \frac{\Delta}{2}(\mathbf{A}x_n + \mathbf{B}u_n + \mathbf{A}x_{n+1} + \mathbf{B}u_{n+1})$$

$$\iff x_{n+1} - \frac{\Delta}{2}\mathbf{A}x_{n+1} = x_n + \frac{\Delta}{2}\mathbf{A}x_n + \frac{\Delta}{2}\mathbf{B}(u_{n+1} + u_n)$$

$$(*) \iff (\mathbf{I} - \frac{\Delta}{2}\mathbf{A})x_{n+1} = (\mathbf{I} + \frac{\Delta}{2}\mathbf{A})x_n + \Delta\mathbf{B}u_{n+1}$$

$$(*) u_{n+1} \stackrel{\Delta}{\simeq} u_n$$

Discretization

$$x' = \mathbf{A}x + \mathbf{B}u$$

$$y = \mathbf{C}x$$

Trapezoid method:
$$x_{n+1} - x_n = \frac{1}{2}\Delta(f(t_n) + f(t_{n+1})) \quad \Delta = t_{n+1} - t_n$$

$$x_{n+1} = x_n + \frac{\Delta}{2}(\mathbf{A}x_n + \mathbf{B}u_n + \mathbf{A}x_{n+1} + \mathbf{B}u_{n+1})$$

$$\iff x_{n+1} - \frac{\Delta}{2}\mathbf{A}x_{n+1} = x_n + \frac{\Delta}{2}\mathbf{A}x_n + \frac{\Delta}{2}\mathbf{B}(u_{n+1} + u_n)$$

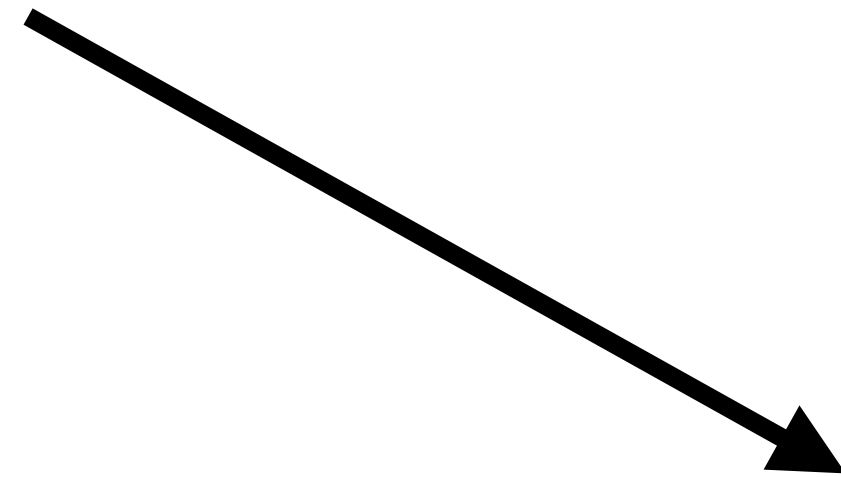
$$(*) \iff (\mathbf{I} - \frac{\Delta}{2}\mathbf{A})x_{n+1} = (\mathbf{I} + \frac{\Delta}{2}\mathbf{A})x_n + \Delta\mathbf{B}u_{n+1}$$

$$(*) \ u_{n+1} \stackrel{\Delta}{\simeq} u_n \quad \iff x_{n+1} = (\mathbf{I} - \frac{\Delta}{2}\mathbf{A})^{-1}(\mathbf{I} + \frac{\Delta}{2}\mathbf{A})x_n + (\mathbf{I} - \frac{\Delta}{2}\mathbf{A})^{-1}\Delta\mathbf{B}u_{n+1}$$

Discretization

$$x' = \mathbf{A}x + \mathbf{B}u$$

$$y = \mathbf{C}x$$



$$\bar{\mathbf{A}} = (\mathbf{I} - \frac{\Delta}{2}\mathbf{A})^{-1}(\mathbf{I} + \frac{\Delta}{2}\mathbf{A})$$

$$\bar{\mathbf{B}} = (\mathbf{I} - \frac{\Delta}{2}\mathbf{A})^{-1}\Delta\mathbf{B}$$

$$\bar{\mathbf{C}} = \mathbf{C}$$

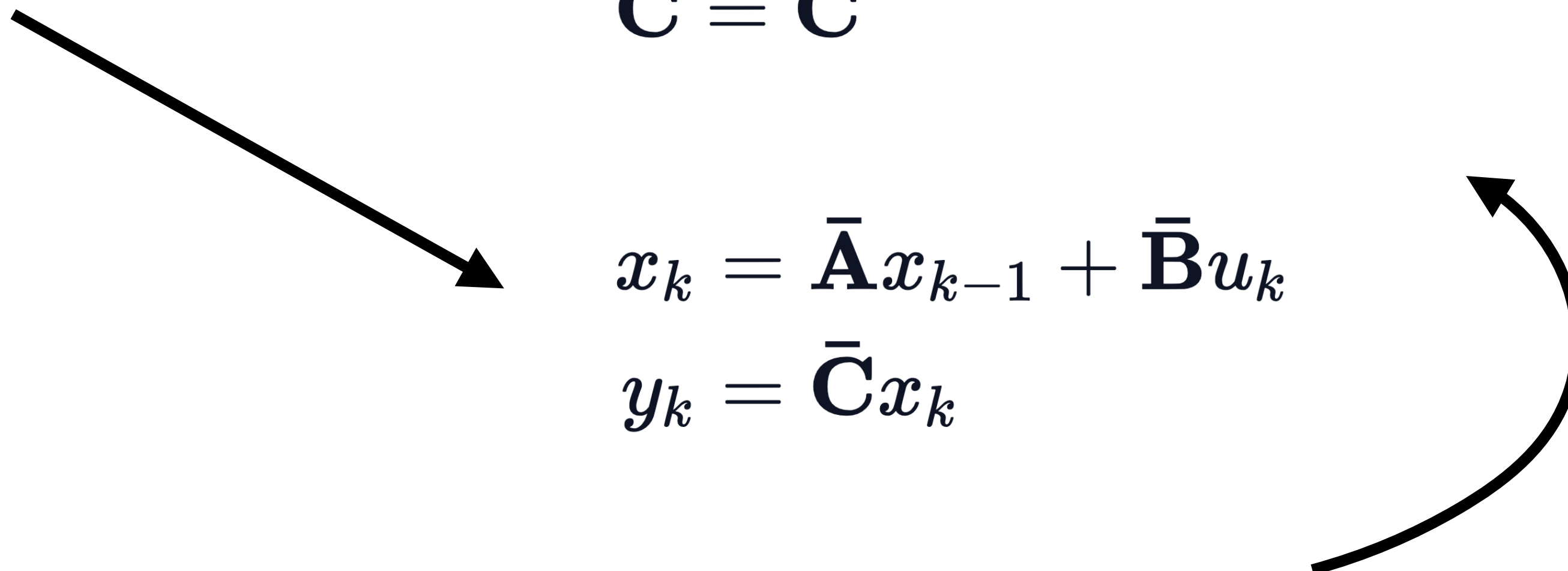
$$x_k = \bar{\mathbf{A}}x_{k-1} + \bar{\mathbf{B}}u_k$$

$$y_k = \bar{\mathbf{C}}x_k$$

Discretization

$$\begin{aligned}x' &= \mathbf{A}x + \mathbf{B}u \\ y &= \mathbf{C}x\end{aligned}$$

$$\begin{aligned}\bar{\mathbf{A}} &= (\mathbf{I} - \frac{\Delta}{2}\mathbf{A})^{-1}(\mathbf{I} + \frac{\Delta}{2}\mathbf{A}) \\ \bar{\mathbf{B}} &= (\mathbf{I} - \frac{\Delta}{2}\mathbf{A})^{-1}\Delta\mathbf{B} \\ \bar{\mathbf{C}} &= \mathbf{C}\end{aligned}$$


$$\begin{aligned}x_k &= \bar{\mathbf{A}}x_{k-1} + \bar{\mathbf{B}}u_k \\ y_k &= \bar{\mathbf{C}}x_k\end{aligned}$$

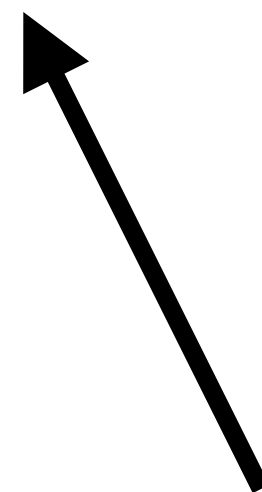
This is called **S4** (Structured State Space sequence model)

Recursive view

SSM

$$x_k = \bar{\mathbf{A}}x_{k-1} + \bar{\mathbf{B}}u_k$$

$$y_k = \bar{\mathbf{C}}x_k$$



Does not have non-linearity!

RNN

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

$$y_t = W_{hy}h_t$$

Convolutional view of an SSM

$$x_k = \bar{\mathbf{A}}x_{k-1} + \bar{\mathbf{B}}u_k$$

$$y_k = \bar{\mathbf{C}}x_k$$

Step 1: $x_0 = \bar{\mathbf{B}}u_0$

Step 2: $x_1 = \bar{\mathbf{A}}x_0 + \bar{\mathbf{B}}u_1 = \bar{\mathbf{A}}\bar{\mathbf{B}}u_0 + \bar{\mathbf{B}}u_1$

Step 3: $x_2 = \bar{\mathbf{A}}x_1 + \bar{\mathbf{B}}u_2 = \bar{\mathbf{A}}(\bar{\mathbf{A}}\bar{\mathbf{B}}u_0 + \bar{\mathbf{B}}u_1) + \bar{\mathbf{B}}u_2 = \bar{\mathbf{A}}^2\bar{\mathbf{B}}u_0 + \bar{\mathbf{A}}\bar{\mathbf{B}}u_1 + \bar{\mathbf{B}}u_2$

x_k is a linear function of $(u_0, u_1, \dots, u_k)$

Convolutive view of an SSM

$$x_k = \bar{\mathbf{A}}x_{k-1} + \bar{\mathbf{B}}u_k$$

$$y_k = \bar{\mathbf{C}}x_k$$

Step 1: $x_0 = \bar{\mathbf{B}}u_0$

Step 2: $x_1 = \bar{\mathbf{A}}x_0 + \bar{\mathbf{B}}u_1 = \bar{\mathbf{A}}\bar{\mathbf{B}}u_0 + \bar{\mathbf{B}}u_1$

Step 3: $x_2 = \bar{\mathbf{A}}x_1 + \bar{\mathbf{B}}u_2 = \bar{\mathbf{A}}(\bar{\mathbf{A}}\bar{\mathbf{B}}u_0 + \bar{\mathbf{B}}u_1) + \bar{\mathbf{B}}u_2 = \bar{\mathbf{A}}^2\bar{\mathbf{B}}u_0 + \bar{\mathbf{A}}\bar{\mathbf{B}}u_1 + \bar{\mathbf{B}}u_2$

Step 1: $y_0 = \bar{\mathbf{C}}x_0 = \bar{\mathbf{C}}\bar{\mathbf{B}}u_0$

Step 2: $y_1 = \bar{\mathbf{C}}x_1 = \bar{\mathbf{C}}(\bar{\mathbf{A}}\bar{\mathbf{B}}u_0 + \bar{\mathbf{B}}u_1) = \bar{\mathbf{C}}\bar{\mathbf{A}}\bar{\mathbf{B}}u_0 + \bar{\mathbf{C}}\bar{\mathbf{B}}u_1$

Step 3: $y_2 = \bar{\mathbf{C}}x_2 = \bar{\mathbf{C}}(\bar{\mathbf{A}}^2\bar{\mathbf{B}}u_0 + \bar{\mathbf{A}}\bar{\mathbf{B}}u_1 + \bar{\mathbf{B}}u_2) = \bar{\mathbf{C}}\bar{\mathbf{A}}^2\bar{\mathbf{B}}u_0 + \bar{\mathbf{C}}\bar{\mathbf{A}}\bar{\mathbf{B}}u_1 + \bar{\mathbf{C}}\bar{\mathbf{B}}u_2$

$$\bar{\mathbf{K}}_k = (\bar{\mathbf{C}}\bar{\mathbf{B}}, \bar{\mathbf{C}}\bar{\mathbf{A}}\bar{\mathbf{B}}, \dots, \bar{\mathbf{C}}\bar{\mathbf{A}}^k\bar{\mathbf{B}})$$

When to use each view

Continuous view:

- ✓ Handles continuous data (audio signals, time series)
- ✗ Extremely slow for both training and inference

When to use each view

Continuous view:

- ✓ Handles continuous data (audio signals, time series)
- ✗ Extremely slow for both training and inference

Recursive view:

- ✓ Natural inductive bias for sequential data, and in principle unbounded context
- ✓ Efficient inference
- ✗ Slow learning (lack of parallelism)
- ✗ Gradient vanish or explosion for long sequences

When to use each view

Continuous view:

- ✓ Handles continuous data (audio signals, time series)
- ✗ Extremely slow for both training and inference

Recursive view:

- ✓ Natural inductive bias for sequential data, and in principle unbounded context
- ✓ Efficient inference
- ✗ Slow learning (lack of parallelism)
- ✗ Gradient vanish or explosion for long sequences

Convolutional view:

- ✓ Efficient (parallelizable) training
- ✗ Slow in autoregressive contexts
(must recalculate entire input for each new data point)
- ✗ Fixed context size

When to use each view

Continuous view:

- ✓ Handles continuous data (audio signals, time series)
- ✗ Extremely slow for both training and inference

Recursive view:

- ✓ Natural inductive bias for sequential data, and in principle unbounded context
- ✓ Efficient inference
- ✗ Slow learning (lack of parallelism)
- ✗ Gradient vanish or explosion for long sequences

Convolutional view:

- ✓ Efficient (parallelizable) training
- ✗ Slow in autoregressive contexts
(must recalculate entire input for each new data point)
- ✗ Fixed context size

Slow powering on A

$$\bar{\mathbf{K}}_k = (\bar{\mathbf{C}}\bar{\mathbf{B}}, \bar{\mathbf{C}}\bar{\mathbf{A}}\bar{\mathbf{B}}, \dots, \bar{\mathbf{C}}\bar{\mathbf{A}}^k\bar{\mathbf{B}})$$

- We need to compute $\bar{\mathbf{A}}^k$ fast
- Easiest way is to make $\bar{\mathbf{A}}^k$ diagonal (class of Normal matrices)

$$\bar{\mathbf{A}}^k = \begin{bmatrix} \lambda_1^k & 0 & \cdots & 0 \\ 0 & \lambda_2^k & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \lambda_n^k \end{bmatrix}$$

Slow powering on $\mathbf{A}$

$$\bar{\mathbf{K}}_k = (\bar{\mathbf{C}}\bar{\mathbf{B}}, \bar{\mathbf{C}}\bar{\mathbf{A}}\bar{\mathbf{B}}, \dots, \bar{\mathbf{C}}\bar{\mathbf{A}}^k\bar{\mathbf{B}})$$

- We need to compute $\bar{\mathbf{A}}^k$ fast
- Easiest way is to make $\bar{\mathbf{A}}^k$ diagonal (class of Normal matrices)

$$\bar{\mathbf{A}}^k = \begin{bmatrix} \lambda_1^k & 0 & \cdots & 0 \\ 0 & \lambda_2^k & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \lambda_n^k \end{bmatrix}$$

- However, initialization matters a lot here.

HiPPO

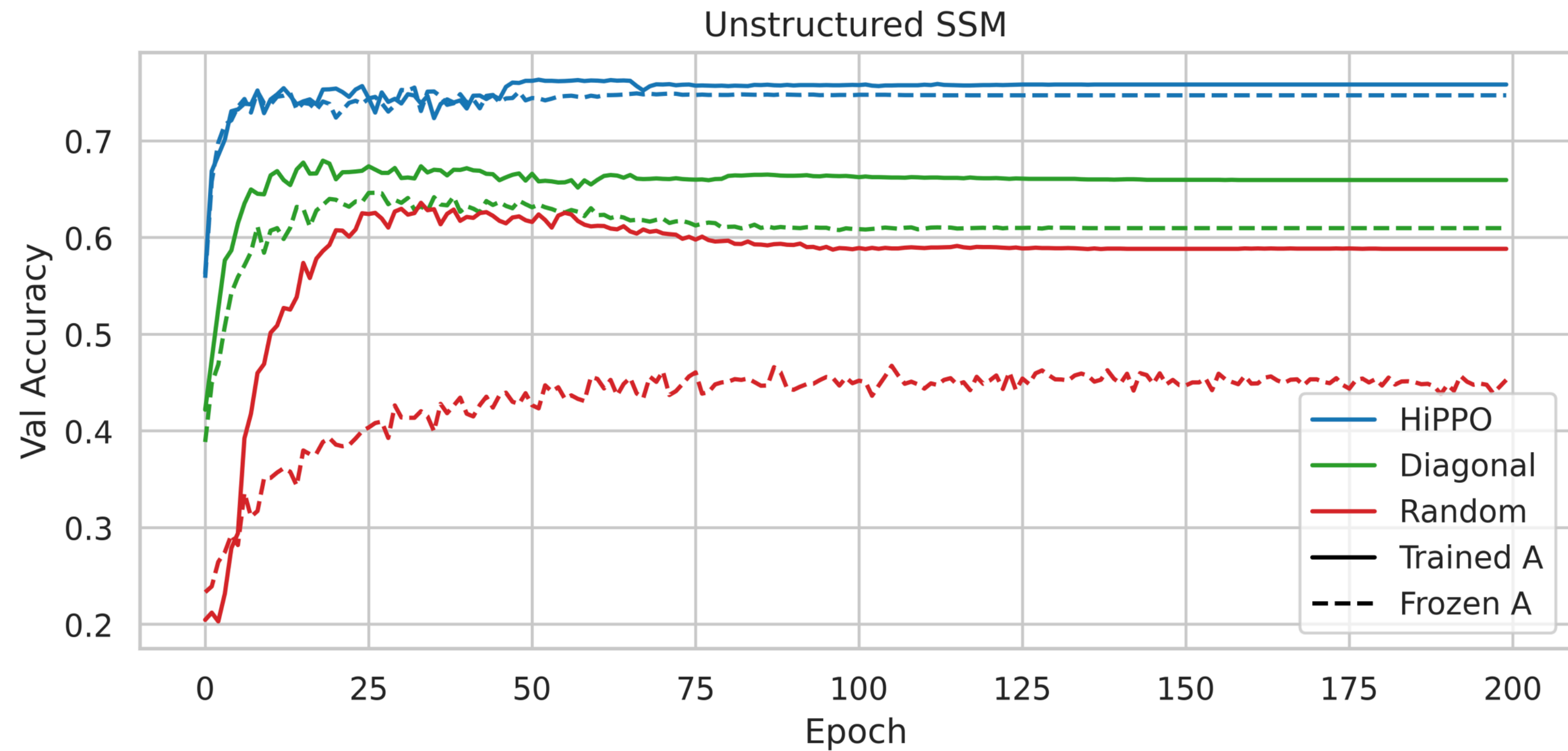
High-Order Polynomial Projection Operator

- Inits $\bar{\mathbf{A}}^k$ with NPLR (Normal Plus Low Rank) matrix
- Boosts quality from 60% to 98% on the MNIST sequential benchmark

$$\mathbf{A} = \begin{bmatrix} 1 & & & & & & & & \\ -1 & 2 & & & & & & & \\ 1 & -3 & 3 & & & & & & \\ -1 & 3 & -5 & 4 & & & & & \\ 1 & -3 & 5 & -7 & 5 & & & & \\ -1 & 3 & -5 & 7 & -9 & 6 & & & \\ 1 & -3 & 5 & -7 & 9 & -11 & 7 & & \\ -1 & 3 & -5 & 7 & -9 & 11 & -13 & 8 & \\ \vdots & & & & & & & & \ddots \end{bmatrix}$$
$$\Rightarrow \mathbf{A}_{nk} = \begin{cases} (-1)^{n-k}(2k+1) & n > k \\ k+1 & n = k \\ 0 & n < k \end{cases}$$

HiPPO

High-Order Polynomial Projection Operator



CIFAR-10 classification accuracy

HiPPO

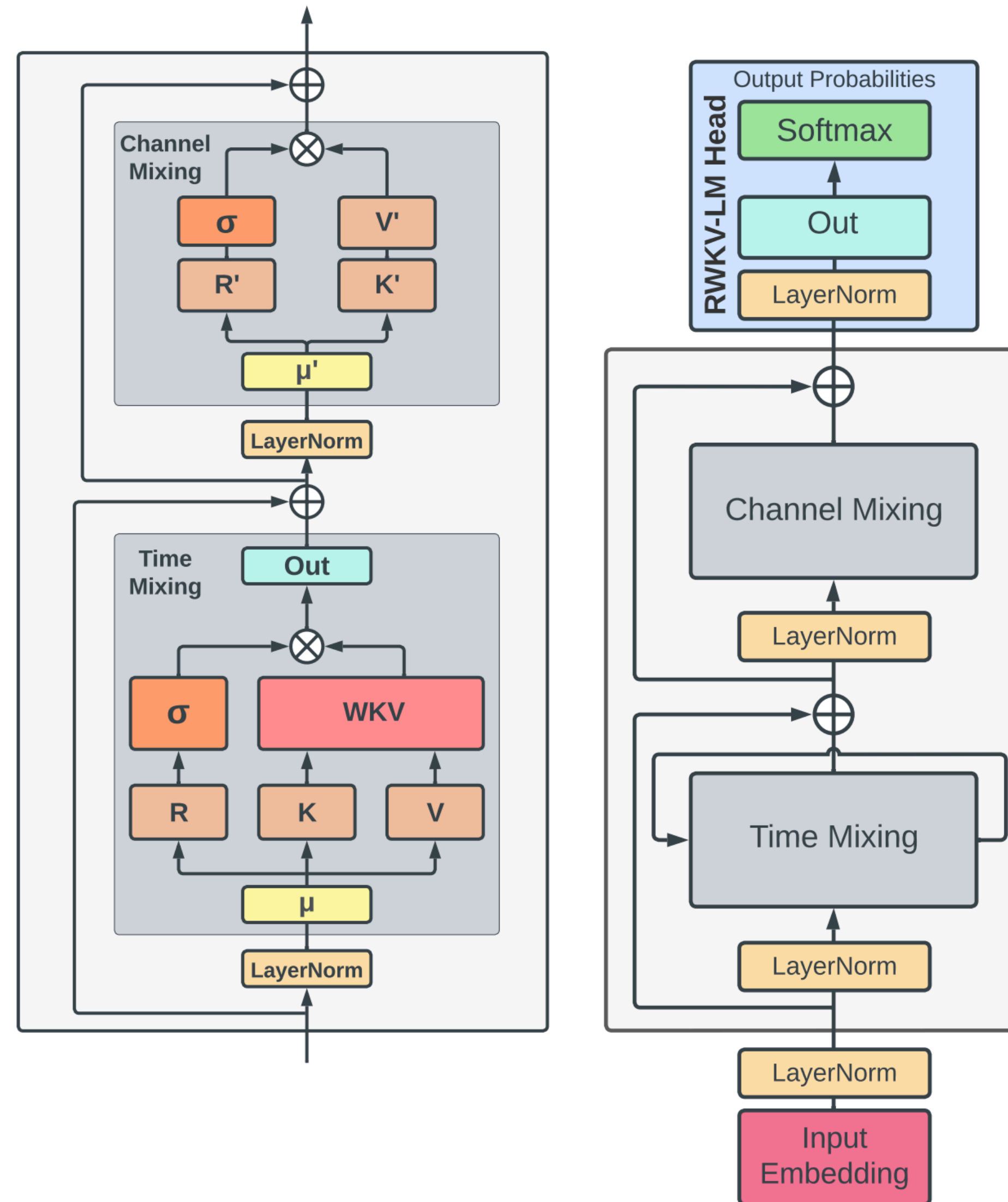
High-Order Polynomial Projection Operator

MODEL	LISTOPS	TEXT	RETRIEVAL	IMAGE	PATHFINDER	PATH-X	AVG
Transformer	36.37	64.27	57.46	42.44	71.40	✗	53.66
Reformer	<u>37.27</u>	56.10	53.40	38.07	68.50	✗	50.56
BigBird	36.05	64.02	59.29	40.83	74.87	✗	54.17
Linear Trans.	16.13	<u>65.90</u>	53.09	42.34	75.30	✗	50.46
Performer	18.01	65.40	53.82	42.77	77.05	✗	51.18
FNet	35.33	65.11	59.61	38.67	<u>77.80</u>	✗	54.42
Nyströmformer	37.15	65.52	<u>79.56</u>	41.58	70.94	✗	57.46
Luna-256	37.25	64.57	79.29	<u>47.38</u>	77.72	✗	<u>59.37</u>
S4	59.60	86.82	90.90	88.65	94.20	96.35	86.09

Long Range Arena benchmark

RWKV

Transformer layer is splitted in
Time-mixing and **Channel-mixing**



RWKV

Time-mixing

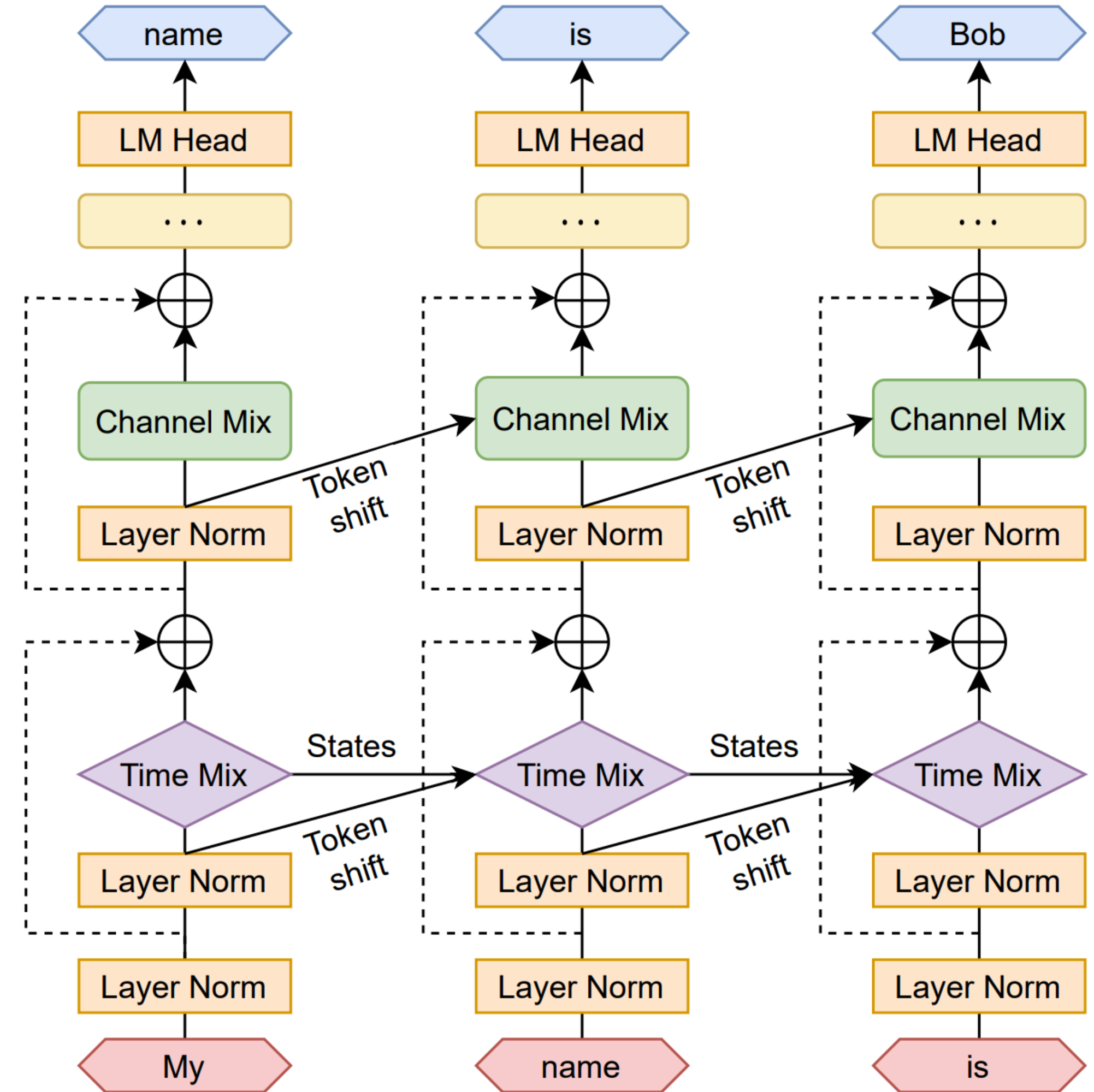
$$r_t = W_r \cdot (\mu_r \odot x_t + (1 - \mu_r) \odot x_{t-1})$$

$$k_t = W_k \cdot (\mu_k \odot x_t + (1 - \mu_k) \odot x_{t-1})$$

$$v_t = W_v \cdot (\mu_v \odot x_t + (1 - \mu_v) \odot x_{t-1})$$

$$wkv_t = \frac{\sum_{i=1}^{t-1} e^{-(t-1-i)w+k_i} \odot v_i + e^{u+k_t} \odot v_t}{\sum_{i=1}^{t-1} e^{-(t-1-i)w+k_i} + e^{u+k_t}}$$

$$o_t = W_o \cdot (\sigma(r_t) \odot wkv_t)$$



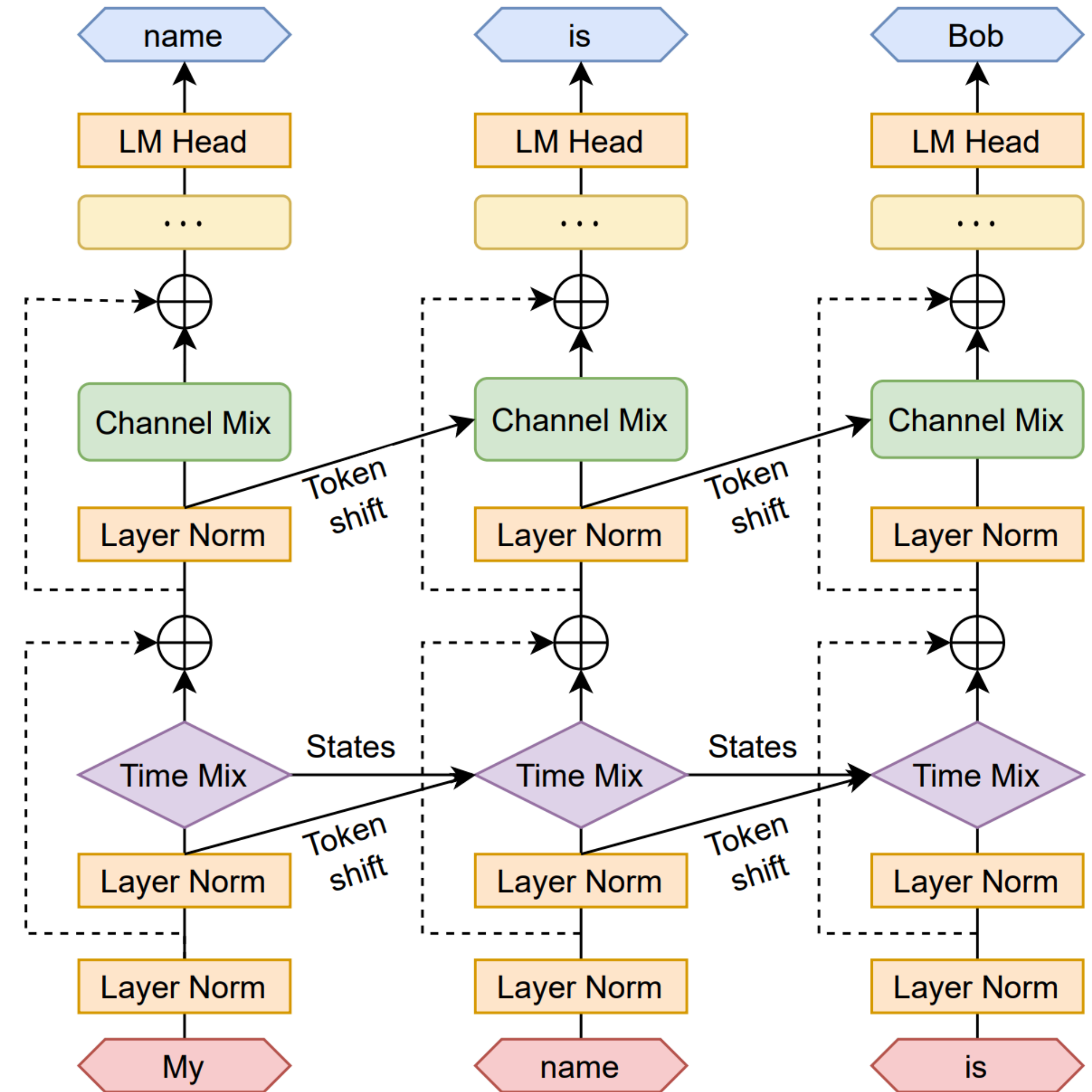
RWKV

Channel-mixing

$$r'_t = W'_r \cdot (\mu'_r \odot x_t + (1 - \mu'_r) \odot x_{t-1})$$

$$k'_t = W'_k \cdot (\mu'_k \odot x_t + (1 - \mu'_k) \odot x_{t-1})$$

$$o'_t = \sigma(r'_t) \odot (W'_v \cdot \max(k'_t, 0)^2)$$



RWKV: results

